CMPT 201 Assignment 3: Group assignment

This assignment is worth 6% of your final grade

Overview

In this assignment you will implement a data processing and compressing module:

- implement an application that can read a data file in CSV format and create a Database (a set of Data Structures) to store it
- implement an application that can interrogate and modify the data stored in the Database
- implement an application to compress the data in the Database for efficient transmission over a network
- create a dashboard which allows users to interact with your implementations
- test your programs extensively, including automated rules for your tests (we want to run the same tests that you have!).
  # Deliverables

This assignment is to be delivered in two parts:

A3-Milestone#1 Due Week 13 in your designated lab period. You must have importDB() and exportDB() implemented. Your exportDB() must reproduce the imported CSV file. To verify this, you must demonstrate to your lab instructur that a system diff does not produce any errors. If only importDB is implemented, you can earn a maximum of 4%. Worth 5%.

A3-Final Submission Due April 10th at 23:59. See the "Submission" section for details.
Assignment Description

In an effort to increase transparency and accessibility, the City of Edmonton maintains and gives the public access to a wide range of datasets. One of these datasets is a Public Picnic Table Location dataset. Information about it can be found here. The dataset can be downloaded in several formats. For the purposes of this assignment we will assume a comma-separated values (.csv) file format. For more info on CSV see Wikipedia entry. The dataset is provided in the A3 package on meskanas.

Each record in the dataset consists of a value corresponding to the following attributes:

Column Name	Description	Type
Id	Unique Site Identifier for all picnic tables in a given area	Number
Table Type	Overall description of picnic table	Text
Surface Material	Basic material used for the picnic table top	Text
Structural Material	Basic material used to build the picnic table	Text
Street / Avenue	Closest street or Avenue the picnic table is near-by	Text
Neighbourhood Id	Unique Neighbourhood Identifier	Number
Neighbourhood Name	Name of the Neighbourhood the picnic table is found in	Text
Ward	Name of the Ward the picnic table is found in	Text
Latitude	Latitude Coordinate for picnic table	Number
Longitude	Longitude Coordinate for picnic table	Number
Location	General spatial point	Text

Design and implement a Database module for efficient storage and retrieval of the above dataset. Your module must be able to read the data from a CSV file (similar to the one attached to the assignment) and store it in a set of tables as defined below. You must store the records of the above dataset in a relevant data structure. In your **design document** justify your choice of data structure and explain the benefits and drawbacks of your choice.

After inspecting the dataset, you will find that some attributes are discrete i.e. they have a countable set of values. Instead of storing these attributes as is in the **main data structure** (we will come back to this in a moment), we would like to codify each. For instance, in PicnicTableSmall.csv you will find three different values for Table Type: Other Table, Square Picnic Table and Round Picnic Table. Therefore, each table type will have a corresponding integer value (code): 0, 1 and 2 respectively. These integer values are assigned in the order in which the type appears as you are reading in the CSV. For instance, the tableType table might look like this:

id	type
0	Other Table
1	Round Picnic Table
2	Square Picnic Table

As a result of using this strategy, you will need the following 4 tables in addition to a picnicTable table (more on this below):

Table Name	Description
tableType Lookup Table	contains a code and a value for each possible entry in the Table Type column of the dataset
surfaceMaterial Lookup Table	contains a code and a value for each possible entry in the Surface Material column of the dataset
structuralMaterial Lookup Table	contains a code and a value for each possible entry in the Structural Material column of the dataset
neighbourhood Lookup Table	contains a code and a value for each possible entry in the Neigbourhood Name column of the dataset. The code must be the one used in the Neighbourhood Id field of the record

You've heard some talk about a **main data structure**. This data structure, picnicTable table, will contain each record from the picnic table CSV that is read in. You can safely assume that the format of the file will be the same. However, your program must not crash if the file doesn't exist or is empty. Each entry in the picnicTable table will contain the following members:

Member Name	Description
table Id	A unique id corresponding to the table. This is a new id to be created by your application
site Id	the id of the site of the picnic table – corresponds to the information in the Id column of the record
tableTypeId	This code must correspond to an entry in the lookup table which holds the same information as the Table Type column of the record
surfaceMaterial Id	This code must correspond to an entry in the lookup table which holds the same information as the Surface Material column of the record
structuralMaterial Id	This code must correspond to an entry in the lookup table which holds the same information as the Structural Material column of the record
streetAvenue	The information in the Street/Avenue column of the record
neighbhd Id	The information in the Neighbourhood Id column of the record
ward	The ward column of the record
latitude	The information in the latitude column of the record. This field may be represented as a string in the picnicTable.
longitude	The information in the longitude column of the record. This field may be represented as a string in the picnicTable.

Important Note

The above lookup tables should be populated simultaneously with the picnicTable table. They are not to be populated manually by searching the CSV for unique column values first and then hardcoding these values in. Otherwise, a severe penalty will apply.

Once the information is properly stored in the relevant tables mentioned above, we would like to be able to interact with your implementation. We have provided you with a public API **DB.h** containing a set of function prototypes for you to implement in a corresponding **DB.c**. In addition, we have provided a private header **DB_impl.h** for any additional supporting functions. Feel free to add more modules as you see fit. We encourage you to do so to foster modularity and separation of concerns.

Interactive Program

In order to effectively test your public API, we would like you to create a program called **dashboard** that takes in either a CSV file or a BIN file in Unix-style command line arguments:

```
./dashboard  -c  PicnicTable.csv
./dashboard  -b  PicnicTable.bin
```

The -c option indicates that the file to be read is a CSV file and the -b option indicates that a compressed binary file is to be read. In either case, upon reading the file, the tables should be populated and ready for user interaction via a dashboard. It is safe to assume that the CSV file will be in the correct format and will be similar to the ones attached to the assignment (although the size may be larger or smaller). However, since compression techniques might differ between developers, you must supply a **PicnicTable.bin** as part of your submission folder. If it is not provided, it will be assumed that this portion of the assignment is incomplete.

Your program must continuously prompt the user for a numerical option until the user opts-out. Please refer to the sample session below (all results are based on PicnicTableSmall.csv):

```
$ ./dashboard -c PicnicTablesSmall.csv

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 2
```

```
Enter member code (1.TT  2.SM   3.StM   4.NID   5.NN   6.W): 1

Enter Value: Round Picnic Table

table type <round picnic table> appeared in the table 1 time(s)

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 3

Enter a criteria to sort by (1.TT  2.SM   3.StM   4.NID   5.NN   6.W): 6

DB exported to ward.csv to verify

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 4

Enter a numerical table entry to edit: 857

Enter member code (1.TT  2.SM   3.StM): 3

Enter Value: Wood

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 1

Enter a filename: out.csv

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 5

Enter a criteria to report by (1. Neighborhood   2. Ward): 1

carter crest:
80,round picnic table,metal,metal,carter crest road nw,5130,carter crest,ward 09,53.4644092853387,-113.575319969328,"(53.46446

dunluce:
126,square picnic table,wood,metal,162 avenue nw,3180,dunluce,ward 03,53.6236218606825,-113.524484955603,"(53.6236218606825, -

jamieson place:
13,square picnic table,wood,aggregate,189a street nw,4220,jamieson place,ward 05,53.4902460605314,-113.649696091432,"(53.49024

laurier heights:
461,other table,wood,aggregate,83 avenue nw,3270,laurier heights,ward 05,53.5164666961262,-113.56908689592,"(53.5164666961262,

mill woods park:
248,square picnic table,wood,aggregate,66 street nw,6510,mill woods park,ward 11,53.4550270455295,-113.436047948875,"(53.45502

river valley gold bar:
```

```
246,square picnic table,wood,metal,goldbar park road nw,6620,river valley gold bar,ward 08,53.5580583877554,-113.40590156789,'

river valley hermitage:
295,square picnic table,wood,metal,hermitage park road nw,2630,river valley hermitage,ward 04,53.5787170918049,-113.369696667

river valley laurier:
857,square picnic table,wood,metal,sir wilfred laurier park road nw,3382,river valley laurier,ward 05,53.5099726873459,-113.55

river valley victoria:
140,square picnic table,wood,wood,river valley road nw,1200,river valley victoria,ward 06,53.5328743816844,-113.517189914531,'
140,square picnic table,wood,wood,river valley road nw,1200,river valley victoria,ward 06,53.5328744794465,-113.517214128796,'

westmount:
479,square picnic table,wood,aggregate,124 street nw,3440,westmount,ward 06,53.5523433466681,-113.535635399607,"(53.552343346

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 5

Enter a criteria to report by (1. Neighborhood   2. Ward): 2

ward 03:
126,square picnic table,wood,metal,162 avenue nw,3180,dunluce,ward 03,53.6236218606825,-113.524484955603,"(53.6236218606825, -

ward 04:
295,square picnic table,wood,metal,hermitage park road nw,2630,river valley hermitage,ward 04,53.5787170918049,-113.369696667

ward 05:
461,other table,wood,aggregate,83 avenue nw,3270,laurier heights,ward 05,53.5164666961262,-113.56908689592,"(53.5164666961262,
13,square picnic table,wood,aggregate,189a street nw,4220,jamieson place,ward 05,53.4902460605314,-113.649696091432,"(53.49024
857,square picnic table,wood,metal,sir wilfred laurier park road nw,3382,river valley laurier,ward 05,53.5099726873459,-113.55

ward 06:
140,square picnic table,wood,wood,river valley road nw,1200,river valley victoria,ward 06,53.5328743816844,-113.517189914531,'
140,square picnic table,wood,wood,river valley road nw,1200,river valley victoria,ward 06,53.5328744794465,-113.517214128796,'
479,square picnic table,wood,aggregate,124 street nw,3440,westmount,ward 06,53.5523433466681,-113.535635399607,"(53.552343346

ward 08:
246,square picnic table,wood,metal,goldbar park road nw,6620,river valley gold bar,ward 08,53.5580583877554,-113.40590156789,'

ward 09:
80,round picnic table,metal,metal,carter crest road nw,5130,carter crest,ward 09,53.4644092853387,-113.575319969328,"(53.46446

ward 11:
248,square picnic table,wood,aggregate,66 street nw,6510,mill woods park,ward 11,53.4550270455295,-113.436047948875,"(53.45502

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 6

Enter filename: out.bin

1. Export Database
2. Count Entries
3. Sort By
4. Edit Entry
5. Report
6. Compress database
7. Exit

option: 7
```

The same functionality should apply if the database was read from the compressed binary file PicnicTable.bin.

Important Notes and Hints

Global Variables

You will use a single global variable (Db, as declared in the header file DB.h) in this assignment. **You must not use any other global variables**.

Compressing the Database

Before writing each table to the **binary file**, it must be proceeded by its length. In other words, you must write the length of a table (number of records) to the binary file first and then write each entry in the table to the file. For an efficient compression, you must use bitfields to represent the integer fields in your structs. If you have variable length fields in your structs (such as strings), you may want to write these members after the constant length members. In addition, these variable length fields must be accompanied by their length as well.

Editing Entries

In the edit entry option, the user may enter a new entry value that doesn't exit in the table. In the event that this does happen, you must maintain database integrity. For example, when editing the table type member to a value that does not exist in the tableType lookup table, the tableType lookup table, must be updated to contain the newly created value.

Testing

We will test your data processing module implementation under the assumption that it can handle any number of entries (as long as memory is available), but they are all coming from a csv file in the given format. However, your program must not crash if a file doesn't exist. A graceful exit will suffice in the event this happens.

You must implement the data processing module, test it appropriately, and then report your effort in a **testing strategy document**.

A collection of your regression test files, must be in a directory called **testing**, which will also contain your testing strategy document. Your testing strategy document should describe what and how you tested, and why it's sufficient. Don't forget, for this assignment, we want to be able to easily run the tests that you ran, so be sure to automate their use in your makefile (this can and should include at least one main program that is only for testing your module).

Design Document

A design document called **design.txt** must describe the design decisions you made. A good design document should record the important decisions you made while analyzing the assignment requirements and how you were able to accomplish them via your implementaion. In particular, it should contain the reasons behind your decisions.

Part of your design document will be the issues list, which identifies the issues that you encountered during the assignment, and how they were resolved.

NOTE: This document is best built from the ground up. Document your decisions and choices along the way to ensure coverage of all pertinent details. This is not to be completed all at once near the deadline.

Group Work

Each group will be composed of 3 to 4 students.

You must create a Git repository and host it privately on GitHub. The members of the team can cooperatively work on the assignment. Your lab instructor may ask access to the repository for grading purposes. Failure to do comply can cost you marks.

Add your lab instructor as a reporter member to your project.

We only require one submission per group. Please speak with your group members and designate a group lead that will be delegated to this task. In addition, within 24 hours past the assignment deadline, you must send an email to your lab instructor indicating the contribution of each group member (percentage).

For instance, if Dhara, Ardy and Hanan were in a group, Hanan might send an email to her lab instructor indicating the following contributions:

- Hanan 30%
- Dhara 35%

- Ardy 35%

NOTE:

- The percentages must add up to a 100%
- If the actual contributions vary greatly between team members, the lab instructor may scale individual marks within a team. Otherwise, all group members receive the same mark by default.

Attached Files

- DB.h
- DB.c
- DB_impl.h
- DB_impl.c
- dashboard.c
- PicnicTableSmall.csv (in the dataset directory)
- PicnicTable.csv (in the dataset directory)

Grading

Criterion	%
Milestone 1	5%
DB.c module tests	65%
dashboard.c	10%
Design document	4%
testing directory (including testing strategy document)	5%
makefile	4%
Documentaion and program design	5%
Git repository created and used	2%

Submission

Your A3 directory must contain the following files:

- DB.h
- DB.c
- DB_impl.h
- DB_impl.c
- dashboard.c (main program)
- Any additional source and header files
- PicnicTable.bin
- makefile (**make all** should create dashboard and **make tests** should run all tests)
- testing directory (includes a testing strategy document)
- design document

Your makefile must also contain a clean and tar target